

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI



MACHINE LEARNING AND DATA MINING II
LABWORK 4: CLASSIFICATION II

TẠ HIẾU NAM (22BI13327)

ĐỖ DUY TOÀN (22BI13420)

Lecturer: Dr. Đoàn Nhật Quang

Contents

1	Decision Tree - DT	1
1.1	Select two datasets with labels available	1
1.2	Analyze and pre-process the data	1
1.2.1	Data Analysis	1
1.2.2	Divide the original dataset into two subsets (80% - 20%)	2
1.3	Build a DT for the training subset	2
1.3.1	Theoretical Background and Algorithm Description	2
1.3.2	Implementation Details	3
1.4	Calculate the error of classification	4
1.4.1	Performance Metrics	4
1.4.2	Experimental Results	4
2	Random Forest - RF	6
2.1	Analyze and pre-process the data	6
2.2	Create K = 100 training set and build 1 testing set	6
2.2.1	Theoretical Background: Ensemble Learning	6
2.2.2	Validation of K=100 (Convergence Analysis)	7
2.3	Build an RF and calculate the error of classification	7
2.4	Compare the results of RF and DT	7
2.4.1	Quantitative Comparison	7
2.4.2	Qualitative Analysis and Comment	7

List of Tables

1	Decision Tree Classification Results.	5
2	Random Forest Classification Results (K=100).	7
3	Comparison: Random Forest vs. Decision Tree.	7

List of Figures

1	Class Distribution (Mice Protein)	1
2	Class Distribution (Isolet)	1
3	PCA Projection (Mice Protein)	2
4	PCA Projection (Isolet)	2
5	Structure of the Decision Tree (Depth=3) for Mice Protein.	3
6	Structure of the Decision Tree (Depth=3) for Isolet.	4
7	Confusion Matrix (DT) - Mice Protein	5
8	Confusion Matrix (DT) - Isolet	6
9	Error vs Trees (Mice)	7
10	Error vs Trees (Isolet)	7
11	Top 20 Feature Importance (Mice Protein)	8
12	Top 20 Feature Importance (Isolet)	8

1 Decision Tree - DT

The objective of this section is to investigate the performance of the Decision Tree algorithm, a fundamental non-parametric supervised learning method, on high-dimensional classification tasks.

1.1 Select two datasets with labels available

To evaluate the versatility of the decision tree model, we selected two distinct datasets representing different domains and feature types:

1. **Mice Protein Expression:** A biological dataset comprising 1080 samples from control, trisomic, and drug-treated mice. Each sample is characterized by 77 continuous features representing protein expression levels. The target variable consists of 8 distinct classes based on genotype, behavior, and treatment.
2. **ISOLET (Isolated Letter Speech Recognition):** A speech processing dataset containing 7797 samples of spoken English alphabet letters (26 classes). Each sample is described by 617 continuous features derived from spectral analysis of the audio signal.

1.2 Analyze and pre-process the data

1.2.1 Data Analysis

Before applying any learning algorithm, it is critical to understand the underlying structure of the data.

Class Distribution. We analyzed the distribution of samples across classes to check for imbalance.

- For **Mice Protein** (Figure 1), the classes are distributed relatively evenly, ensuring that the model will not be biased towards any majority class.
- For **ISOLET** (Figure 2), the distribution is perfectly uniform, which is ideal for multi-class classification training.

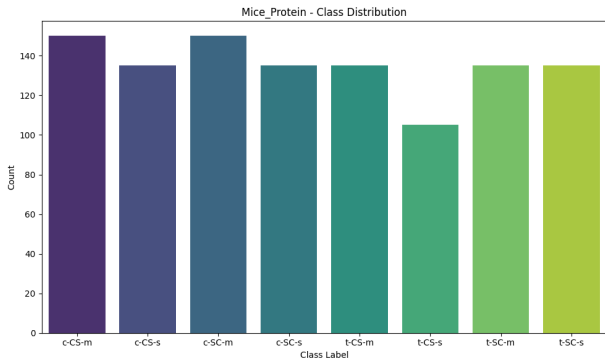


Figure 1: Class Distribution (Mice Protein)

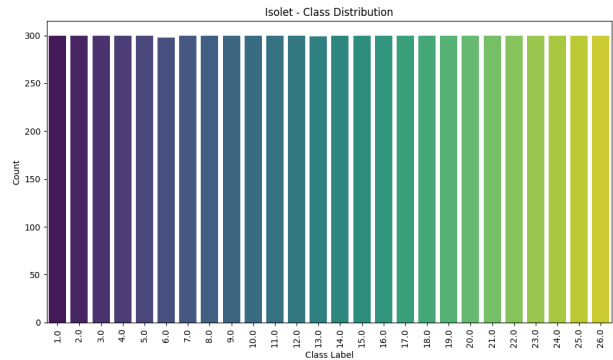


Figure 2: Class Distribution (Isolet)

Dimensionality and Separability (PCA). Given the high dimensionality of the datasets (77 and 617 features), we employed Principal Component Analysis (PCA) to project the data into a 2D space for visualization. Figures 3 and 4 show the first two principal components.

- **Mice Protein:** Distinct clusters are visible, though some overlap exists, suggesting that while some classes are separable, others share similar protein profiles.
- **ISOLET:** The clusters are tightly packed with significant overlap, reflecting the complexity of speech data where different letters may share similar acoustic properties.

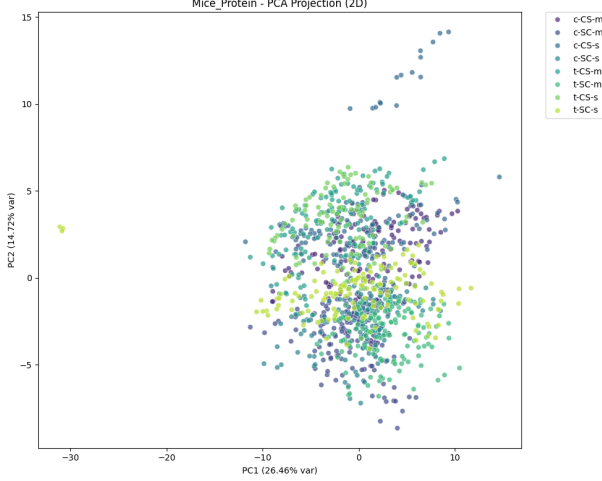


Figure 3: PCA Projection (Mice Protein)

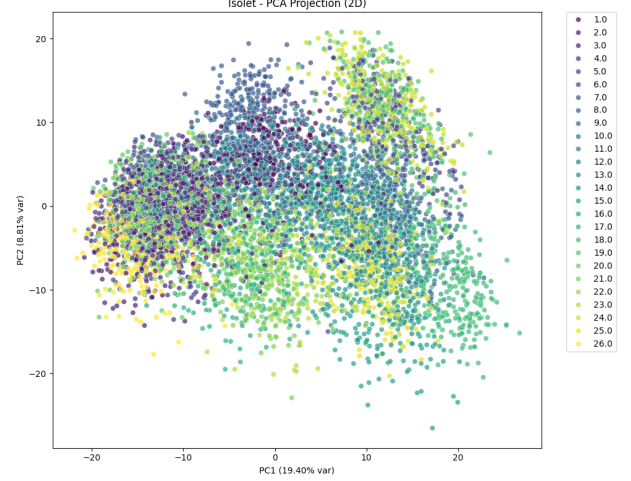


Figure 4: PCA Projection (Isolet)

1.2.2 Divide the original dataset into two subsets (80% - 20%)

To strictly adhere to the hold-out validation protocol, we partitioned the datasets into training and testing subsets using the `train_test_split` function from `sklearn.model_selection`. We used a fixed `random_state=42` to ensure reproducibility.

1. **Training Set (80%):** 864 samples for Mice Protein, 6237 samples for Isolet. Used for model fitting.
2. **Testing Set (20%):** 216 samples for Mice Protein, 1560 samples for Isolet. Used exclusively for evaluation.

Missing values in the Mice Protein dataset were handled via simple imputation (filling with 0) to ensure compatibility with the sklearn implementation.

1.3 Build a DT for the training subset

1.3.1 Theoretical Background and Algorithm Description

A Decision Tree is a hierarchical model that partitions the feature space into rectangular regions. The tree structure consists of:

- **Root Node:** Represents the entire dataset.
- **Internal Nodes:** Represent decision points based on a specific feature threshold (e.g., $Feature_X \leq 0.5$).
- **Leaf Nodes:** Represent the final class assignment.

The core algorithm (CART) recursively splits the data to maximize the purity of the resulting subsets. The quality of a split is determined by an impurity metric.

Gini Impurity. This measures the probability that a randomly selected element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset.

$$G(S) = 1 - \sum_{i=1}^c p_i^2 \quad (1)$$

where S is the dataset at the node, c is the number of classes, and p_i is the proportion of samples belonging to class i . $G = 0$ indicates a perfectly pure node.

Entropy (Information Gain). Alternatively, Entropy measures the uncertainty in the dataset.

$$H(S) = - \sum_{i=1}^c p_i \log_2 p_i \quad (2)$$

The algorithm selects the split that maximizes the **Information Gain**, which is the reduction in Entropy after the split.

1.3.2 Implementation Details

We utilized the `DecisionTreeClassifier` from the `sklearn.tree` package. The model was initialized with default parameters (Gini criterion) and trained on the 80% subset. To visualize the decision logic, we generated pruned trees (Depth=3) as shown in Figures 5 and 6. This visualization helps in interpreting which features are most discriminative at the top levels of the hierarchy.

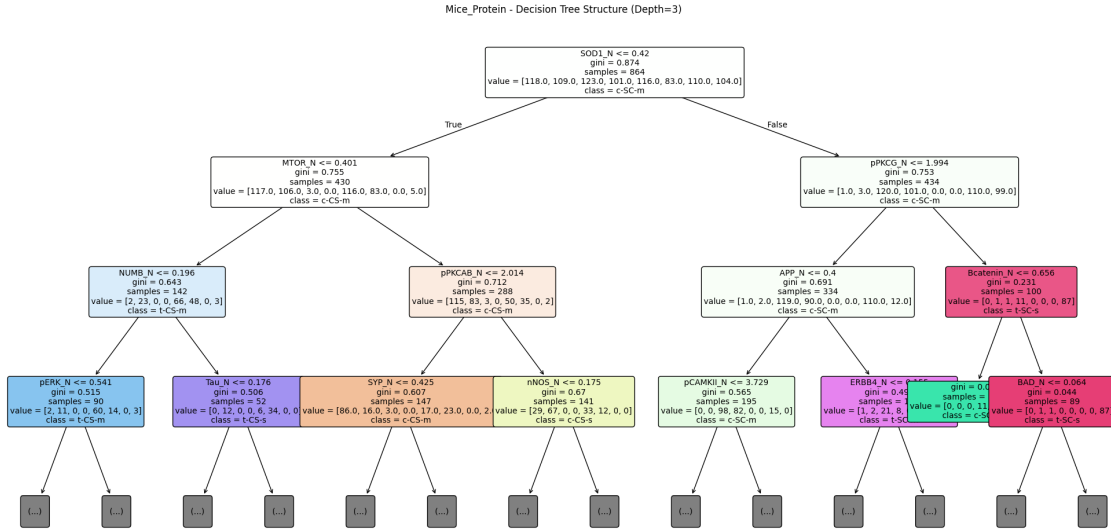


Figure 5: Structure of the Decision Tree (Depth=3) for Mice Protein.

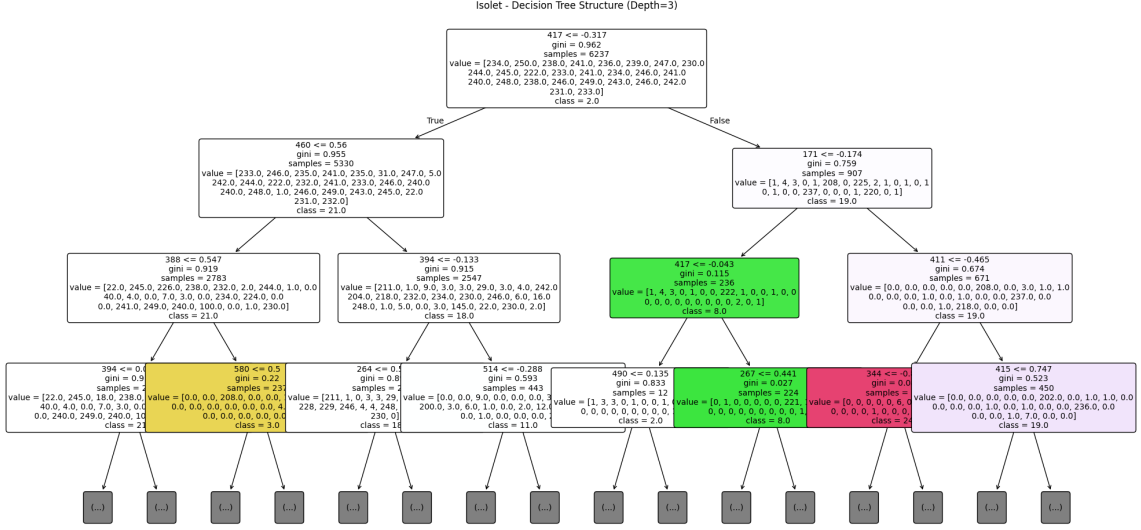


Figure 6: Structure of the Decision Tree (Depth=3) for Islet.

1.4 Calculate the error of classification

1.4.1 Performance Metrics

To provide a comprehensive evaluation, we calculated four key metrics.

Accuracy. The proportion of correctly classified instances.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (3)$$

Precision. The ratio of correctly predicted positive observations to the total predicted positive observations.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (4)$$

Recall (Sensitivity). The ratio of correctly predicted positive observations to the all observations in actual class.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (5)$$

F1-score. The harmonic mean of Precision and Recall.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6)$$

For multi-class problems, Precision, Recall, and F1 are calculated as the weighted average across all classes.

1.4.2 Experimental Results

The performance of the Decision Tree on the test set is summarized in Table 1.

Dataset	Accuracy	Error	Precision	Recall	F1-score
Mice Protein	89.81%	10.19%	0.9017	0.8981	0.8979
ISOLET	81.60%	18.40%	0.8180	0.8160	0.8157

Table 1: Decision Tree Classification Results.

Analysis. For the **Mice Protein** dataset, the model achieved nearly 90% accuracy. The confusion matrix (Figure 7) shows a strong diagonal, but misclassifications occur between biologically similar classes. For **ISOLET**, the accuracy drops to 81.6%. The high dimensionality (617 features) and the complexity of speech signals likely cause the single decision tree to overfit certain noise patterns in the training data, leading to poorer generalization errors (18.4%) on the test set. This highlights the limitation of single estimators.

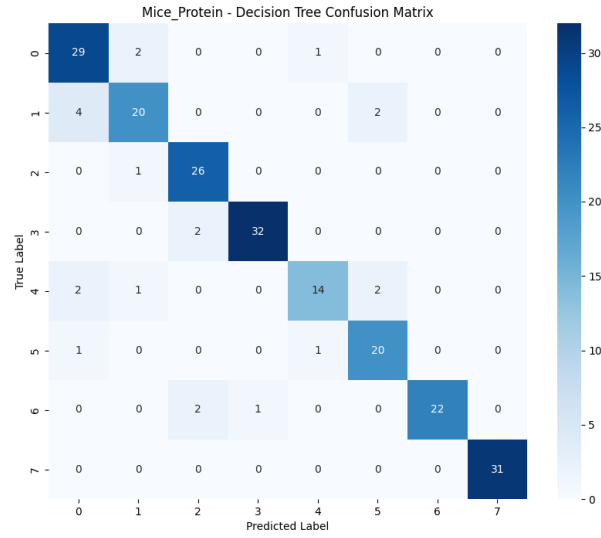


Figure 7: Confusion Matrix (DT) - Mice Protein

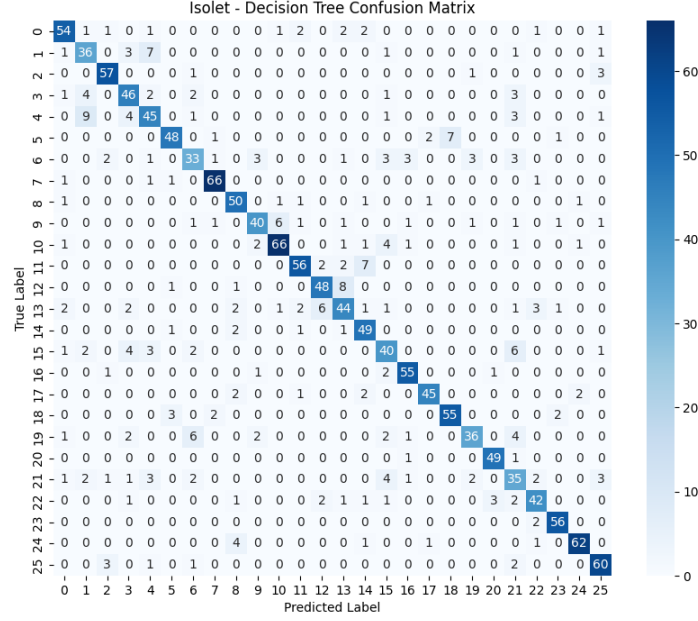


Figure 8: Confusion Matrix (DT) - Isolet

2 Random Forest - RF

2.1 Analyze and pre-process the data

The data analysis and pre-processing steps (including PCA, class distribution analysis, and handling missing values) are identical to those described in Section 1.2, ensuring a fair comparison between the two models on the exact same data splits.

2.2 Create $K = 100$ training set and build 1 testing set

2.2.1 Theoretical Background: Ensemble Learning

Random Forest is an ensemble learning method that improves upon valid Decision Trees by aggregating the predictions of multiple individual trees (K trees). It employs two main techniques to reduce variance:

1. Bagging (Bootstrap Aggregating). For each tree T_k (where $k = 1 \dots K$), we create a new training set D_k by sampling N examples from the original dataset D uniformly and **with replacement**. This means some samples may appear multiple times in D_k , while others are omitted (On average, $\approx 63.2\%$ of unique samples are included).

2. Feature Randomness. When splitting a node during tree construction, instead of searching for the best split among all M features, Random Forest searches for the best split among a random subset of m features (typically $m = \sqrt{M}$). This decorrelates the trees, ensuring that they do not all rely on the same dominant features.

2.2.2 Validation of K=100 (Convergence Analysis)

We constructed $K = 100$ training sets using the bagging technique. To verify that 100 trees are sufficient, we analyzed the **OOB (Out-of-Bag) / Test Error convergence**. Figures 9 and 10 plot the classification error against the number of trees. The error rate drops sharply initially and stabilizes around 40-50 trees. Using $K = 100$ ensures we are well into the stable region, minimizing variance.

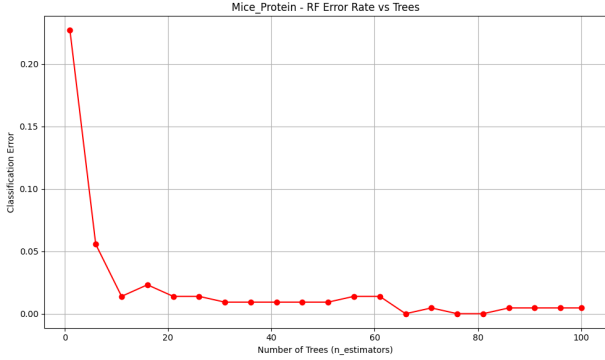


Figure 9: Error vs Trees (Mice)

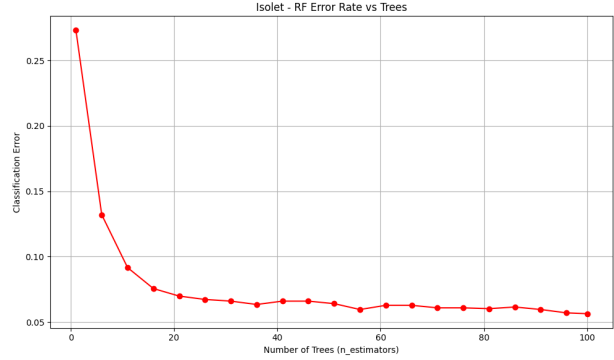


Figure 10: Error vs Trees (Isolet)

2.3 Build an RF and calculate the error of classification

We trained the Random Forest classifier ($K = 100$, `random_state = 42`) on the training subsets and classified the data from the testing subsets using majority voting.

Dataset	Accuracy	Error	Precision	Recall	F1-score
Mice Protein	99.54%	0.46%	0.9955	0.9954	0.9954
ISOLET	94.36%	5.64%	0.9451	0.9436	0.9438

Table 2: Random Forest Classification Results (K=100).

2.4 Compare the results of RF and DT

2.4.1 Quantitative Comparison

Table 3 provides a direct comparison of the two methods.

Dataset	DT Accuracy	RF Accuracy	Improvement	Error Reduction
Mice Protein	89.81%	99.54%	+9.73%	95.5%
ISOLET	81.60%	94.36%	+12.76%	69.3%

Table 3: Comparison: Random Forest vs. Decision Tree.

2.4.2 Qualitative Analysis and Comment

Performance Improvement. Random Forest drastically outperforms the single Decision Tree. For the Mice Protein dataset, the error was virtually eliminated (reduced by 95.5%). For Isolet, accurate classification increased by nearly 13 percentage points.

Bias-Variance Tradeoff. The primary reason for this improvement is the reduction of **Variance**. Single Decision Trees are prone to overfitting—they grow deep and memorize noise in the training data. By averaging 100 trees trained on bootstrapped data, Random Forest smooths out these idiosyncrasies. The decision boundary becomes more robust and generalizable.

Feature Importance. Random Forest also allows us to inspect which features drive the decisions. Figures 11 and 12 show the top 20 most important features. For Mice Protein, certain key proteins dominate the classification, aligning with biological expectations. For Isolet, specific spectral frequencies are more critical for distinguishing phonemes.

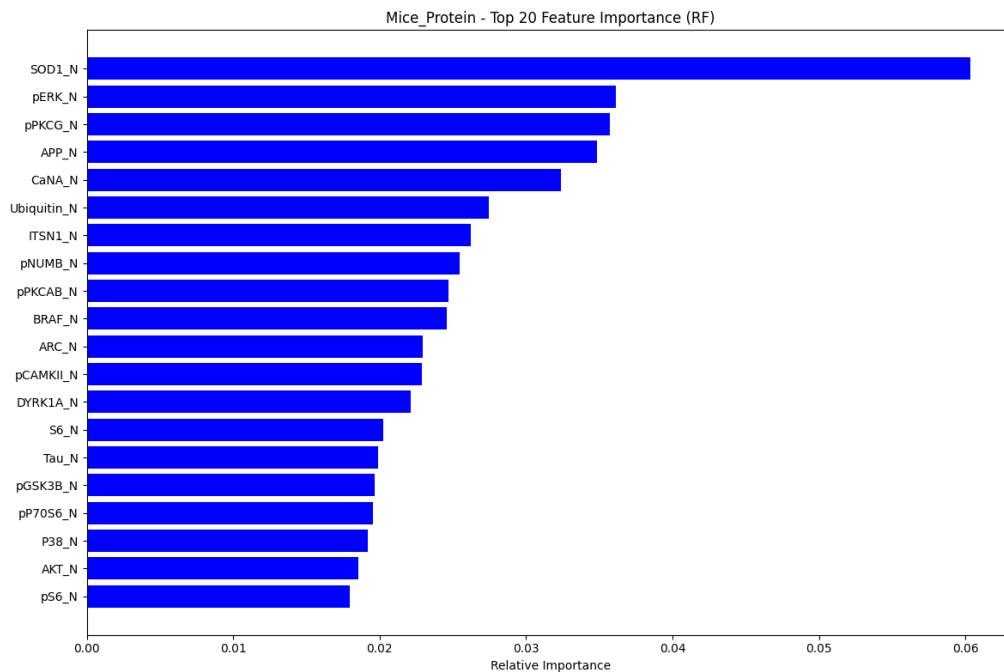


Figure 11: Top 20 Feature Importance (Mice Protein)

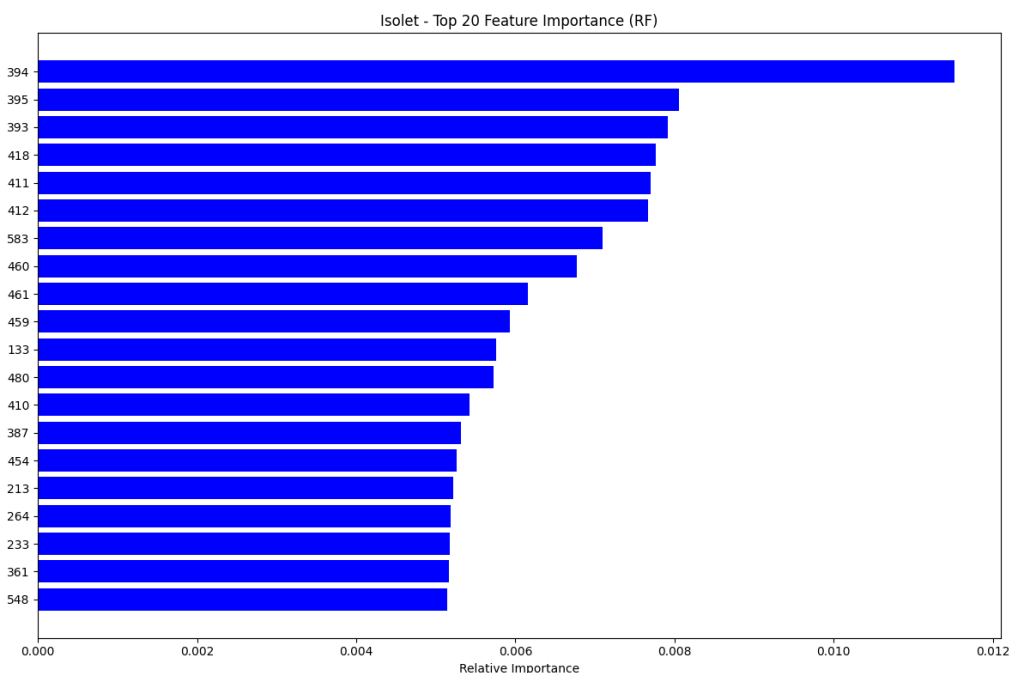


Figure 12: Top 20 Feature Importance (Isolet)